

MEDIATEK

MT8676 Yocto T-Box User Manual

版本: 1.0
出版日期: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	Liuyutian Liu	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 T-Box 5

 1.1 概述 5

 1.1.1 简单介绍 5

 1.1.2 T-Box 缩略词 5

 1.2 架构/进程概述 6

 1.2.1 T-Box 架构 6

 1.2.2 T-Box API 说明 6

 1.3 常见问题/故障排除 18

 1.3.1 SIM/CALL/SMS/Telephony 驻网相关 18

 1.3.2 Data 相关 18

 1.3.3 网络相关 19

 1.3.4 IMS 相关 19

附件一 附加条款 20

图片目录

图 1-1. T-Box 架构 6

表格目录

表 1-1. 缩略词 5

表 1-2. Modem 状态及 IMEI 接口说明 6

表 1-3. SIM 接口说明 7

表 1-4. Telephony 网路接口说明 8

表 1-5. 数据接口说明 10

表 1-6. 电话接口说明 12

表 1-7. 短信接口说明 13

表 1-8. IMS 接口说明 14

表 1-9. Ecall control 接口说明 14

表 1-10. AT 黑名单接口说明 17

表 1-11. MIPC Keep alive 接口说明 17

1 T-Box

1.1 概述

1.1.1 简单介绍

本文介绍 Yocto T-Box 架构、API 使用及注意事项。

1.1.2 T-Box 缩略词

表 1-1.缩略词

缩略词	全称	释义
API	Application Programming Interface	应用程序编程接口
APN	Access Point Name	接入点
T-Box	Telematics-BOX	车联网控制单元

1.2 架构/进程概述

1.2.1 T-Box 架构

Hypervisor Dual System Tbox Architecture

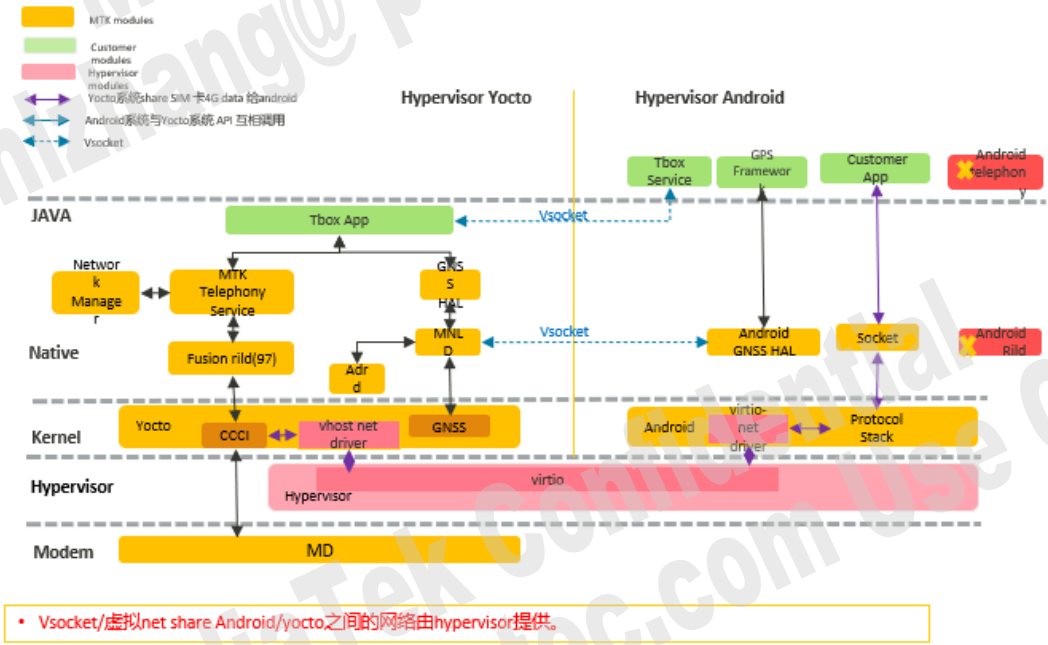


图 1-1. T-Box 架构

1.2.2 T-Box API 说明

1.2.2.1 Modem 状态/IMEI 接口及调用时序说明

T-Box API 除 ML_GetModemStat, 其他均需要基于 modem ready 的情况下才能正常工作。

表 1-2. Modem 状态及 IMEI 接口说明

接口/结构体	描述
Int ML_GetModemStat(char *stat, uint32_t statlen)	获取 modem 状态, stat 返回字符串“ready”表示 modem 已正常启动成功。
Int ML_Getimei(char* imei, size_t imeiLen)	获取平台 IMEI int 返回值 On success, 0 is returned. On error, -1 is returned.

1.2.2.2 SIM 接口及调用时序说明

SIM 接口除 ML_Sim_GetCardStatus，其他均需要基于 SIM present 的情况下才能正常获取到值。

表 1-3. SIM 接口说明

接口/结构体	描述
<pre>typedef enum { E_ML_SIM_CARD_STATE_UNKNOWN = 0xB01, /**< Card state unknown. */ E_ML_SIM_CARD_STATE_ABSENT = 0xB02, /**< Card is absent. */ E_ML_SIM_CARD_STATE_PRESENT = 0xB03, /**< Card is present. */ E_ML_SIM_CARD_STATE_ERROR_UNKNOWN = 0xB04, /**< Unknown error state. */ E_ML_SIM_CARD_STATE_ERROR_POWER_DOWN = 0xB05, /**< Power down. */ E_ML_SIM_CARD_STATE_ERROR_POLL_ERROR = 0xB06, /**< Poll error. */ E_ML_SIM_CARD_STATE_ERROR_NO_ATR_RECEIVED = 0xB07, /**< Failed to receive an answer to reset. */ E_ML_SIM_CARD_STATE_ERROR_VOLT_MISMATCH = 0xB08, /**< Voltage mismatch. */ E_ML_SIM_CARD_STATE_ERROR_PARITY_ERROR = 0xB09, /**< Parity error. */ E_ML_SIM_CARD_STATE_ERROR_SIM_TECHNICAL_PROBLEMS= 0xB0A, /**< Card returned technical problems. */ }E_ML_SIM_CARD_STATE_TYPE_T; /**< Card state. */</pre>	SIM 卡状态结构体。
Int32_t ML_Sim_GetCardStatus(ML_SIM_CARD_STATUS_INFO_T *pvsSimStatus)	获取 SIM 状态。pvsSimStatus 返回 “present” 表示 SIM 卡在位。 int 返回值 On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetICCID(char *iccid, size_t iccidLen)	获取 SIM 卡 ICCID。 int 返回值 On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetMsisdn(char *msisdn, size_t msisdnLen)	获取 SIM 卡 Msisdn。 int 返回值 On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetImsi(char *imsi, size_t imsilen)	获取 SIM 卡 IMSI。 int 返回值 On success, 0 is returned. On error, -1 is returned.
Int ML_SIM_GetMccMnc(char *mcc, size_t mccLen, char*mnc, size_t mncLen)	获取 SIM 卡 mccmnc。 int 返回值 On success, 0 is returned. On error, -1 is returned.

1.2.2.3 Telephony 网络接口及调用时序说明

表 1-4. Telephony 网路接口说明

接口/结构体	描述
<pre>typedef enum { E_ML_NW_PS_UNKNOWN = 0x00, E_ML_NW_PS_ATTACHED = 0x01, E_ML_NW_PS_DETACHED = 0x02, }E_ML_NW_PS_REG_STATE_TYPE_T;</pre>	PS 网络注册状态结构体。
<pre>typedef enum { E_ML_NW_CS_UNKNOWN = 0x00, E_ML_NW_CS_ATTACHED = 0x01, E_ML_NW_CS_DETACHED = 0x02, }E_ML_NW_CS_REG_STATE_TYPE_T;</pre>	CS 网络注册状态结构体。
<pre>int ML_GetNetState(E_ML_NW_PS_REG_STATE_TYPE_T * ps, E_ML_NW_CS_REG_STATE_TYPE_T * cs)</pre>	获取 CS/PS 网络注册状态。 int 返回值 On success, 0 is returned. On error, -1 is returned. 注意：此接口需在识卡后调用
<pre>int32_t ML_GetOperatorCode (uint8_t * op_code)</pre>	获取运营商名称信息。op_code 为 1 表示 SIM 运营商是中国联通, 为 0 表示中国移动。 int 返回值 On success, 0 is returned. On error, -1 is returned. 注意：此接口需在识卡后调用
<pre>typedef enum { E_ML_NW_RAT_GSM = 0, E_ML_NW_RAT_UTRAN = 2, E_ML_NW_RAT_GSMW_EGPRS = 3, E_ML_NW_RAT_UTRANW_HSDPA = 4, E_ML_NW_RAT_UTRANW_HSUPA = 5, E_ML_NW_RAT_UTRANW_HSDPA_AND_HSUPA = 6, E_ML_NW_RAT_E_UTRAN = 7, #ifdef SPM_TELEPHONY_NR_SUPPORT E_ML_NW_RAT_NR = 8, #endif }E_ML_NW_RADIO_ACCESS_TYPE_T;</pre>	网络制式类型结构体。
<pre>int ML_GetRadioAccessType (E_ML_NW_RADIO_ACCESS_TYPE_T * access_type)</pre>	获取驻上网络的制式类型。 int 返回值 On success, 0 is returned. On error, -1 is returned. 注意：此接口需在识卡后调用
<pre>int ML_GetSignalStrength (int8_t * sig_level)</pre>	获取信号强度等级。sig_level: 0~4, 信号等级阈值参考 AOSP default 划分。 int 返回值 On success, 0 is returned. On error, -1 is returned.

接口/结构体	描述
	注意：此接口需在识卡后调用
int ML_SetAirplaneMode (uint8_t on_off)	设置开关飞行模式。on_off 为 1 表示开飞行模式， on_off 为 0 表示关飞行模式。 int 返回值 On success, 0 is returned. On error, -1 is returned. 注意：此接口需在识卡后调用
typedef void (*ml_signal_strength_cb_t)(ML_SignalStrength *state);	信号强度回调函数。
int ML_SignalStreghInit (ml_signal_strength_cb_t evt_cb)	注册信号强度回调。 int 返回值 On success, 0 is returned. On error, -1 is returned.
typedef enum { E_ML_AUTO = 0, E_ML_2GONLY = 1, E_ML_3GONLY = 2, E_ML_4GONLY = 3, #ifdef SPM_TELEPHONY_NR_SUPPORT E_ML_5GONLY = 4, E_ML_5GAUTO = 5, E_ML_5G4GAUTO = 6, #endif }ml_nw_net_mode_e;	网络模式结构体。
int ML_SetNetMode (ml_nw_net_mode_e mode)	设置网络模式。 int 返回值 On success, 0 is returned. On error, -1 is returned.
int ML_GetNetMode (ml_nw_net_mode_e * mode)	获取网络模式。 int 返回值 On success, 0 is returned. On error, -1 is returned.

1.2.2.4 数据接口及调用时序说明

表 1-5. 数据接口说明

接口/结构体	描述
<pre>typedef struct { unsigned char profile_idx; ml_apn_pdp_type_e pdp_type; ml_apn_auth_proto_e auth_proto; char apn_name[ML_APN_NAME_SIZE]; char username[ML_APN_USERNAME_SIZE]; char password[ML_APN_PASSWORD_SIZE]; }ml_apn_info_s</pre>	APN 信息结构体 profile_idx: 1 – 8 (1: 公网 ID; 2-8: 私网 ID, 至多支持 7 路私网 APN) pdp_type: IPV4/IPV6/IPV4V6, 详情参考 ml_apn_pdp_type_e enum auth_proto: default/none/pap/chap/pap_chap apn_name: APN 名字
<pre>int ML_APN_Get (unsigned char profile_idx, ml_apn_info_s * apn)</pre>	根据 profile_idx 获取对应的 APN 信息。 参数: 输入: unsigned char profile_idx 输出: ml_apn_info_s *apn APN 初始化流程: 开机识别到卡后, mtktelephonyervice 判断 /data/vendor/telephony/apn.db 是否存在, 若不存在则根据 /vendor/etc/apns_conf.xml 生成/data/vendor/telephony/apn.db。再通过卡的 mccmnc 从 apn.db 里获取 SIM 卡的 APN, 设定 APN 到 modem。 注意: 此接口需在识卡后调用, 即 ML_Sim_GetCardStatus() 返回 present 后。
<pre>int ML_APN_Set (const ml_apn_info_s * apn)</pre>	设定 APN 到 modem 和 APN database (/data/vendor/telephony/apn.db) 参数: 输入: const ml_apn_info_s * apn 注意: 此接口需在识卡后调用
<pre>int ML_DataCallInit (ml_data_call_evt_cb_t evt_cb)</pre>	注册 data call 状态变化 callback 函数, 当 data call 状态发生变化时回调 evt_cb 函数。 参数: 输入: ml_data_call_evt_cb_t evt_cb

接口/结构体	描述
<pre>int ML_DataCallStart (const ml_data_call_s * data_call, ml_data_call_error_e * err)</pre>	建立公网/私网 PDN 连接。 同步接口，下发建立 PDN 命令到 modem，modem 返回后，此接口才返回。 参数： 输入：const ml_data_call_s * data_call 输出：ml_data_call_error_e * err
<pre>int ML_DataCallStop (char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_error_e * err)</pre>	断开公网/私网 PDN 连接。 同步接口，下发建立 PDN 命令到 modem，modem 返回后，此接口才返回。 参数： 输入：char profile_idx 输入：ml_data_call_ip_family_e ip_family 输出：ml_data_call_error_e * err
<pre>int ML_DataCallStart_Ext (const ml_data_call_s * data_call, ml_data_call_error_e * err)</pre>	建立公网/私网 PDN 连接。 异步接口，下发建立 PDN 命令后直接返回，不等 modem 执行结果。建立 PDN 的状态通过 ML_DataCallInit 注册的 callback 函数回调给 APP。 参数： 输入：const ml_data_call_s * data_call 输出：ml_data_call_error_e * err
<pre>int ML_DataCallStop_Ext (char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_error_e * err)</pre>	断开公网/私网 PDN 连接。 异步接口，下发断开 PDN 命令后直接返回，不等 modem 执行结果。断开 PDN 的状态通过 ML_DataCallInit 注册的 callback 函数回调给 APP。 参数： 输入：char profile_idx 输入：ml_data_call_ip_family_e ip_family 输出：ml_data_call_error_e * err
<pre>int ML_getDataCallReason (int32_t profile_idx, ML_DataCallFailCause * reason)</pre>	PDN 建立失败时，获取 failcause。 参数： 输入：profile_idx

接口/结构体	描述
	输入/输出： reason
Int ML_DataCallInfoGet(char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_info_s *info, ml_data_call_error_e *err)	根据 profile_idx, ip_family 参数: 输入： char profile_idx 输入： ml_data_call_ip_family_e ip_family 输出： ml_data_call_info_s *info 输出： ml_data_call_error_e *err

1.2.2.5 电话接口及调用时序说明

电话接口均需要基于网络已注册的情况下调用。

表 1-6. 电话接口说明

接口/结构体	描述
typedef struct { int32_t CallId; char PhoneNum[32]; E_ML_VCALL_STATE_TYPE_T State; int32_t call_end_reason; } ML_VCALL_INFO_T;	电话信息结构体。 CallId: 当前电话 index; PhoneNum[32]: 对端电话号码; State: 电话状态; call_end_reason: 电话挂断;
typedef void (*ML_VCALL_MSGCB_T) (ML_VCALL_INFO_T *pvsMsg);	电话回调函数。
Int32_t ML_VcallInit(ML_VCALL_MSGCB_T cb_func)	注册电话回调函数。 来电时触发回调函数，来电信息通过回调函数返回。 参数: 输出： ML_VCALL_MSGCB_T cb_func;
Int ML_VcallStart(const char *PhoneNumber)	拨出电话。 参数: 输入： const char *PhoneNumber;
Int ML_VcallAnswer(void)	接听电话。
Int ML_VcallEnd (void)	挂断电话。

1.2.2.6 短信接口及调用时序说明

短信接口均需要基于网络已注册的情况下调用。

表 1-7. 短信接口说明

接口/结构体	描述
<pre>typedef enum { E_ML_SMS_FORMAT_GSM_7BIT = 0, E_ML_SMS_FORMAT_BINARY_DATA = 1, E_ML_SMS_FORMAT_UCS2 = 2, E_ML_SMS_FORMAT_IRA = 3, }E_ML_SMS_FORMAT_T;</pre>	短信格式枚举定义。
<pre>typedef struct { E_ML_SMS_FORMAT_T format; char PhoneNum[ML_SMS_MAX_ADDR_LENGTH]; int32_t SmsDataLen; char SmsData[ML_SMS_MAX_MT_MSG_LENGTH]; } ML_SMS_INFO_T;</pre>	短信信息结构体。
<pre>typedef void (*ML_SMS_RXMSGCB_T) (ML_SMS_INFO_T *pvsMsg);</pre>	短信回调函数。
<pre>Int ML_SmsInit(ML_SMS_RXMSGCB_T cb_func)</pre>	注册短信回调函数。 接收短信时触发回调函数，短信信息通过回调函数返回。 参数： 输出： ML_SMS_RXMSGCB_T cb_func;
<pre>Int32_t ML_Sms_Sent(const ML_SMS_INFO_T *pvsSms)</pre>	发送短信。 异步接口，发送短信请求发送后会异步等待 modem 返回响应信息，不会卡住线程。 参数： 输入： const ML_SMS_INFO_T *pvsSms;

1.2.2.7 IMS 接口及调用时序说明

表 1-8. IMS 接口说明

接口/结构体	描述
Int ML_EnableIms(uint8_t on_off)	开启/关闭 IMS 功能 传入参数： on_off 为 1 表示开启 IMS 功能。 On_off 为 0，表示关闭 IMS 功能。
Int ML_GetImsRegState(int8_t *reg_state)	获取 IMS 注册状态 传出参数 reg_state 值为 1，表示 IMS 注册上， reg_state 值为 0 则表示 IMS 没有注册上。

1.2.2.8 Ecall 接口及调用时序说明

基于《Ecall Overview》文档中 ecall overall sequence 里的 ECALL 相关的 RIL Request，将以下 API 添加进 MT8676。

表 1-9. Ecall control 接口说明

接口/结构体	描述
typedef struct { int32_t call_id; uint32_t length; unsigned char msd_data[ML_ECALL_MSD_MAX_LENGTH]; }ml_ecall_set_msd;	设置 MSD 时传入结构体参数定义。
typedef struct { int32_t arg_num; int32_t type; char address[128]; }ml_ecall_set_num;	设置 test number 及 reconfig number 时传入结构体参数定义。
typedef enum { ML_EMER_CAT_MANUAL_ECALL = 1, ML_EMER_CAT_AUTO_ECALL = 2, }ml_ecall_category;	发生 Ecall 类别的结构体。
typedef enum { ML_ECALL_TEST = 1, ML_ECALL_EMERGENCY = 2, ML_ECALL_RECONFIG = 3, }ml_ecall_variant;	Ecall 类型的结构体。

接口/结构体	描述
<pre>typedef enum{ ML_DOMAIN_AUTO = 0, /* Automatic mode - LTE(IMS), WG(CS), 1x(C2K) */ ML_DOMAIN_CS_ONLY = 1, /* CS domain only - WG(CS) */ ML_DOMAIN_3GPP_ONLY = 2, /* 3GPP only - LTE(IMS), WG(CS) */ ML_DOMAIN_3GPP2 = 3, /* 3GPP2 only - 1x(C2K) */ ML_DOMAIN_IMS_1xCS = 4, /* IMS and 1x CS only - LTE(IMS), 1x(C2K) */ ML_DOMAIN_CS_1x = 5, /* WG CS and 1x CS only - WG(CS), 1x(C2K) */ ML_DOMAIN_IMS_ONLY = 6, /* only IMS call allowed */ }ml_ecall_domain;</pre>	Ecall 通话的网络制式结构体。
<pre>typedef struct{ ml_ecall_category ecall_cat; ml_ecall_variant ecall_variant; char address[20]; uint32_t length; unsigned char msd_data[ML_ECALL_MSD_MAX_LENGTH]; ml_ecall_domain domain; }ml_ecall_req_msg;</pre>	请求拨出 Ecall 时传入结构体参数定义。
<pre>typedef struct{ int32_t data1; int32_t data2; int32_t data3; int32_t data4; }ml_ecall_pri;</pre>	Ecall 优先级参数结构体。data1>data2>data3>data4。传入参数应为 1 2 3 4，含义分别为： 1：客户设定的 Ecall URI 2：USIM 保存的 Ecall URI 3：客户设定的 ECall 号码 4：USIM 保存的 Ecall 号码
<pre>typedef enum{ E_ML_ECALL_SENDING_START = 1, E_ML_ECALL_SENDING_MSD = 2, E_ML_ECALL_LLACK_RECEIVED = 3, E_ML_ECALL_ALACK_POSITIVE_RECEIVED = 4, E_ML_ECALL_ALACK_CLEARDOWN_RECEIVED = 5, E_ML_ECALL_DIALING = 9, E_ML_ECALL_ALERTING = 10, E_ML_ECALL_ACTIVE = 11,</pre>	Ecall 上报状态的消息类型枚举定义。

接口/结构体	描述
<pre> E_ML_ECALL_DISCONNECTED = 12, E_ML_ECALL_IMS_ACTIVE = 13, E_ML_ECALL_IMS_DISCONNECTED = 14, E_ML_ECALL_ABNORMAL_HANGUP=15, E_ML_ECALL_IMS_MSD_ACK = 20, E_ML_ECALL_IMS_UPDATE_MSD = 21, E_ML_ECALL_IMS_IN_BAND_TRANSFER = 22, E_ML_ECALL_IMS_MSD_NACK = 23, E_ML_ECALL_IMS_SRVCC = 24, E_ML_ECALL_ONLY_DEREGISTRATION = 31, E_ML_ECALL_MAY_DEREGISTER = 32, E_ML_ECALL_PSAP_CALLBACK_START = 40, E_ML_ECALL_PSAP_CALLBACK_IMS_UPDATE_MSD = 41, E_ML_ECALL_T2_TIMEOUT = 52, E_ML_ECALL_T5_TIMEOUT = 55, E_ML_ECALL_T6_TIMEOUT = 56, E_ML_ECALL_T7_TIMEOUT = 57, E_ML_ECALL_UNSPECIFIED = 0xffff, }ML_ECall_Indication; </pre>	
<pre> typedef struct{ ML_ECall_Indication ind; int call_id; } ML_ECALL_IND_T; </pre>	Ecall 上报状态的消息结构体。
<pre> typedef void (*ML_ECALL_MSGCB_T)(ML_ECALL_IND_T *pvsMsg); </pre>	Ecall 上报状态消息的回调函数。
<pre> int32_t ML_EcallIndicationInit(ML_ECALL_MSGCB_T cb_func); </pre>	注册 Ecall 状态变化 callback 函数，当 Ecall 状态发生变化时回调 cb_func 函数。 参数： 输入：ML_ECALL_MSGCB_T cb_func
<pre> int32_t ML_ResetIvs(void); </pre>	重置 modem 的 ecall 状态. 重连 audio。
<pre> int32_t ML_SetMSD(ml_ecall_set_msd* msd); </pre>	设置 ecall 的 msd 数据。 参数： 输入：ml_ecall_set_msd* msd
<pre> int32_t ML_SetTestNumber(ml_ecall_set_num* test_num); </pre>	设置 test number 或者 URI。 参数： 输入：ml_ecall_set_num* test_num

接口/结构体	描述
int32_t ML_SetReconfNumber(ml_ecall_set_num* reconf_num);	设置 reconfig number 或者 URI。 参数： 输入: ml_ecall_set_num* test_num
int32_t ML_MakeFastEcall(ml_ecall_req_msg* msg);	拨出 ecall 同时传入 msd 数据。 参数： 输入: ml_ecall_req_msg* msg
int32_t ML_SetEmsdpri(ml_ecall_pri* pri);	设置 ecall 优先级类别。默认优先级是 “1>3>2>4”。 参数： 输入: ml_ecall_pri* pri

1.2.2.9 休眠唤醒 Modem 相关接口及调用时序说明

表 1-10. AT 黑名单接口说明

接口/结构体	描述
Int ML_SendAT(const char* atCmd, char* finalRsp, uint32_t resp_len, int64_t timeout_ms); Int ML_SetUnsolResponseFilter(ML_UnsolResponseFilter filter)	进入 IPO 前，开启 AT URC 黑名单过滤功能： 1. 调用 ML_SendAT 函数下发 AT 命令：AT+EURCFLT=1 2. 调用 ML_SetUnsolResponseFilter(0) 退出 IPO 后，关闭 AT URC 黑名单过滤功能： 1. 调用 ML_SendAT 函数下发 AT 命令：AT+EURCFLT=0 2. 调用 ML_SetUnsolResponseFilter(0xFF)

表 1-11. MIPC Keep alive 接口说明

接口/结构体	描述
ML 接口：计划基于 MIPC 接口封装 ML 层 API。 struct keepaliveRequest { keepaliveType type; uint32_t sourcePort; uint32_t desinationPort; uint32_t cid; char sourceAddress[68]; char desinationAddress[68]; } struct keepaliveStatus{	通过发送数据包来保持互联网 PDN/PDU 会话处于连接模式，而不唤醒 AP 处理器。 流程： 1. AP 调用 ML_StartKeepalive。回调 keepaliveStatus 状态为 pending，表示等待 AP 发送第一个 UL packet。

接口/结构体	描述
ACTIVE = 0, INACTIVE, PENDING } ML_StartKeepalive(keepaliveRequest * keepalive) ML_StopKeepalive(int cid) ML_InitKeepaliveStatusCb(ml_keepalive_evt_cb_t evt_cb) MIPC interface: MIPC_DATA_KEEPALIVE_REQ/CNF MIPC_DATA_KEEPALIVE_STATUS_IND	2. 等 modem 收到 AP 发送的第一个 UL packet 后，回调 keepaliveStatus 状态为 active，表示 MD 能发送 keep-alive 封包了。 3. 若出现异常，当 modem 最终收不到网络 keep-alive 响应信息后，回调 keepaliveStatus 为 inactive。

1.3 常见问题/故障排除

1.3.1 SIM/CALL/SMS/Telephony 驻网相关

- 如何关闭/开启指定网络类型，参考[FAQ17447] [NW]关闭指定网络类型。
- SIM 测项失败 预先调试：
 - IMEI 初始值是空的，需要先烧录才能获取到。
 - 若 SIM 识别异常，抓取包含开机过程的测试场景 mtklog 协助进一步分析。
- 驻网失败 预先调试：
 - 确认平台有烧录 RF/IMEI，正确安装天线，SIM 卡正确插入且不欠费。
 - 若 (1) 确认后仍然无法驻网，抓取包含开机过程的测试场景 mtklog 协助进一步分析。
- CALL/SMS 失败 预先调试：
 - 使用以下 命令 确认平台是否有成功驻上网，若未驻网成功，先解决驻网问题。
 adb shell /usr/bin/mlclient_test --gtest_filter=MLClientTest.ML_GetNetState
 - 若 (1) 步骤已经驻网成功，抓取测试场景 mtklog 协助进一步分析。

1.3.2 Data 相关

APN 配置文件的修改，[FAQ21414] MTK apns-conf.xml configuration guide。

T-Box 使用的配置文件命名为 apns-conf.xml；代码路径：src/telephonyware/3.1/libvendor-ril/apn/resource/apns-conf.xml；平台中的安装路径：/system/etc/tele/apns-conf.xml

若直接修改/system/etc/tele/apns-conf.xml，需删除 apn.db，重启生效。

1.3.3 网络相关

网络建立成功的检查：

- 1) 网络接口 `up` 并且 IP 地址存在，透过 `ifconfig` 命令确认。
- 2) 网络 IP rule & IP route & DNS 设置，透过 `ip rule/ip route/dumpsys netd/dumpsys dnsresolver` 命令确认。

网络连通性的检查：

手动 ping 某 ip 或某网址是否能 ping 通。

1.3.4 IMS 相关

如何默认开机自动开启或者关闭 IMS？

- (1) 当前系统默认开机自动开启 IMS 功能。
- (2) 可通过改变 `ProjectConfig.mk` 里面 `MTK_VOLTE_SUPPORT` 为 `yes` 或者 `no`，来改变是否支持 IMS。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。